

UNIVERSIDAD NACIONAL DEL ALTIPLANO - PUNO

FACULTAD DE INGENIERÍA MECÁNICA

ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

**ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS**



**PROYECTO FINAL: DESARROLLO DE UNA
APLICACIÓN DE SISTEMA DE RECOMENDACIÓN
HÍBRIDO DE LAPTOPS**

CURSO: SISTEMAS DE RECOMENDACIÓN

PRESENTADO POR:

CARMEN NIEVES APAZA CONDORI

**INFORME TÉCNICO DE APLICACIÓN FUNCIONAL, PIPELINE DE DATOS Y
EVALUACIÓN DE MODELOS**

PUNO - PERÚ

2026

Resumen

El presente proyecto reporta el diseño, desarrollo, implementación y evaluación científica de un **Sistema de Recomendación Híbrido en Dos Niveles** para computadoras portátiles (laptops), concebido para mitigar los problemas de escasez de datos (**data sparsity**) e inicio en frío (**cold start**). La arquitectura integra un **Filtro Estricto por Reglas de Dominio en Cascada** (M_{rules}) en el Nivel 1, seguido de un **Ensamble Ponderado Dinámico** en el Nivel 2 que combina: (i) Teoría de Utilidad Multi-Atributo (**MAUT**) basada en modelos de conocimiento de hardware, (ii) Factorización Matricial **SVD** (**Singular Value Decomposition**) para filtrado colaborativo, y (iii) Filtrado Basado en Contenido mediante similitud de coseno vectorial (**TF-IDF**).

La plataforma web fue desarrollada utilizando **FastAPI** como backend de alta velocidad y un frontend interactivo en **HTML5, CSS3 Vanilla, JavaScript ES6+ y Chart.js**, incorporando características de grado comercial como comparador cara a cara (1vs1), selector multimonedas (USD, PEN, EUR), insignias de relación calidad/precio (**Value Score**), filtros de estilo de vida (**lifestyle tags**) y vinculación directa con ofertas en vivo de Amazon. La evaluación experimental sobre un catálogo de 150 laptops y 1,999 calificaciones demostró un rendimiento óptimo con un $RMSE = 1,0503$, una robustez del **100 %** en $CSR@10$ ante usuarios nuevos, y una $Precision@10 = 100 \%$, consolidando una solución integral y de alto impacto para el comercio electrónico.

Palabras Clave: Sistemas de Recomendación Híbridos, Factorización Matricial SVD, MAUT, Inicio en Frío, FastAPI, Chart.js.

Abstract

This project reports the design, development, implementation, and scientific evaluation of a **Two-Level Hybrid Recommender System** for laptop computers, engineered to mitigate data sparsity and cold start problems. The architecture integrates a **Cascaded Domain Rule Mask** (M_{rules}) at Level 1, followed by a **Dynamic Weighted Ensemble** at Level 2 combining: (i) Multi-Attribute Utility Theory (**MAUT**) based on domain knowledge models, (ii) **SVD** Matrix Factorization for collaborative filtering, and (iii) Content-Based Filtering via vector cosine similarity (**TF-IDF**).

The web platform was built using **FastAPI** as a high-speed backend and an interactive frontend with **HTML5**, **Vanilla CSS3**, **JavaScript ES6+**, and **Chart.js**, incorporating commercial-grade features such as a 1vs1 head-to-head comparison tool, multi-currency selector (USD, PEN, EUR), value score badges, lifestyle filters, and direct Amazon store offer integration. Experimental evaluation over a catalog of 150 laptops and 1,999 ratings demonstrated optimal performance with $RMSE = 1,0503$, **100% CSR@10** robustness under cold start scenarios, and **100% Precision@10**, establishing a high-impact solution for modern e-commerce environments.

Keywords: Hybrid Recommender Systems, SVD Matrix Factorization, MAUT, Cold Start, FastAPI, Chart.js.

Índice

Resumen	I
Abstract	II
Índice General	IV
Índice de Tablas	V
Índice de Figuras	VI
1. Introducción	1
1.1. Contexto del Proyecto	1
1.2. Planteamiento del Problema	1
1.3. Justificación	1
1.4. Objetivos del Proyecto	2
1.4.1. Objetivo General	2
1.4.2. Objetivos Específicos	2
2. Descripción General y Arquitectura del Sistema	3
2.1. Descripción General del Proyecto	3
2.2. Pipeline de Procesamiento de Datos y EDA	3
2.3. Análisis Exploratorio de Datos (EDA) y Preprocesamiento	3
2.4. Arquitectura del Sistema en 2 Niveles	4
2.4.1. Nivel 1: Filtro Estricto por Reglas Lógicas de Dominio	4
2.4.2. Nivel 2: Ensamble Ponderado Dinámico	4
3. Metodología y Fases del Trabajo	4
3.1. Fase 1: Mejora y Formulación del Modelo Híbrido	4
3.1.1. 1. Filtro Estricto por Reglas Lógicas de Dominio	5
3.1.2. 2. Teoría de Utilidad Multi-Atributo (MAUT)	5
3.1.3. 3. Factorización Matricial SVD (Filtrado Colaborativo)	5
3.1.4. 4. Filtrado Basado en Contenido (TF-IDF y Similitud Coseno)	5
3.2. Fase 2: Desarrollo de la Aplicación Web Interactiva	5
3.3. Fase 3: Evaluación Científica del Sistema	6

3.4. Fase 4: Experiencia de Usuario (UX) e Interfaz	7
3.5. Fase 5: Presentación Final y Sustentación	7
4. Resultados y Análisis de Resultados	7
4.1. Resultados de la Evaluación Experimental	7
4.2. Gráficos Científicos de Rendimiento	8
4.3. Evidencias de la Aplicación Funcional	9
4.3.1. 1. Dashboard Principal e Interfaz Híbrida	9
4.3.2. 2. Tarjetas de Recomendación y Vinculación con Amazon	9
4.3.3. 3. Comparador Cara a Cara (Head-to-Head 1vs1)	10
4.3.4. 4. Gráfica de Tendencia e Histórico de Precios (Chart.js)	11
4.3.5. 5. Selector Multimoneda Global (Soles S/)	11
4.3.6. 6. Filtros de Estilo de Vida (Lifestyle Tags)	12
4.3.7. 7. Catálogo Completo y Búsqueda en Tiempo Real	12
4.3.8. 8. Pestaña de Evaluación Científica	13
5. Discusión	13
6. Conclusiones	13
6.1. Logros Alcanzados	13
6.2. Limitaciones del Sistema	14
6.3. Aprendizajes Obtentidos	14
7. Recomendaciones y Trabajo Futuro	15
7.1. Mejoras Futuras	15
7.2. Posibles Extensiones del Sistema	15
8. Referencias Bibliográficas	15
9. Anexos	16
9.1. Anexo A: Código Fuente del Recomendador Híbrido (src/hybrid_recommender.py)	16
9.2. Anexo B: Estructura del Dataset (data/laptops.csv)	17
9.3. Anexo C: Guía de Despliegue y Ejecución	18

Índice de tablas

1.	Comparación experimental cuantitativa de métricas entre modelos.	7
----	--	---

Índice de figuras

1.	Diagrama del Pipeline de Procesamiento de Datos del Recomendador Híbrido.	3
2.	Evaluación de Cobertura de Inicio en Frío ($CSR@10$) comparando SVD vs Recomendador Híbrido.	8
3.	Curvas de Precision@K y Recall@K alcanzadas por el sistema híbrido. . .	8
4.	Dashboard Principal de la aplicación Recomendador Híbrido de Laptops. .	9
5.	Tarjeta de recomendación con desglose de puntaje y enlace a oferta en Amazon.	10
6.	Modal de Comparativa Cara a Cara (1vs1) entre dos computadoras portátiles.	10
7.	Gráfica interactiva de tendencia e histórico de precios en 6 meses con Chart.js.	11
8.	Aplicación adaptada dinámicamente a la moneda Soles Peruanos (PEN S/).	11
9.	Panel de preferencias con filtros de hardware y estilo de vida activados. . .	12
10.	Vista de la pestaña del Catálogo Completo de 150 laptops con buscador. . .	12
11.	Pestaña de Evaluación Científica proyectando métricas y gráficos de rendimiento.	13

1. Introducción

1.1. Contexto del Proyecto

En la era digital actual, el comercio electrónico de bienes tecnológicos experimenta un crecimiento exponencial. Sin embargo, la compra de computadoras portátiles (laptops) representa una decisión de alta complejidad cognitiva para los usuarios debido a la abundancia de especificaciones técnicas heterogéneas (microarquitectura de CPU, núcleos, memoria RAM, aceleradores GPU, potencia de VRAM, pantallas OLED/4K y soporte CUDA).

Los algoritmos de filtrado colaborativo tradicionales (*Collaborative Filtering*) sufren de limitaciones severas en este dominio, principalmente la escasez de datos (*data sparsity*) y el problema del inicio en frío (*cold start*), en el cual usuarios nuevos sin historial previo de calificaciones no pueden recibir sugerencias relevantes.

1.2. Planteamiento del Problema

Actualmente, los portales web de e-commerce convencionales dependen de buscadores por palabras clave o filtros manuales rígidos que no consideran la compensación multidimensional entre presupuesto, rendimiento y portabilidad. Las principales deficiencias identificadas son:

1. **Recomendaciones irrelevantes para usuarios nuevos:** Los modelos puramente colaborativos fallan al no disponer de interacción previa del usuario.
2. **Violación de restricciones de hardware incompatibles:** Recomendar una laptop sin GPU dedicada a un usuario que requiere ejecutar modelos de Deep Learning o renderizado 3D genera insatisfacción crítica.
3. **Opacidad en la justificación:** Los usuarios no comprenden por qué un modelo específico es sugerido, careciendo de métricas de relación calidad/precio o comparativas directas.

1.3. Justificación

El desarrollo de un ****Sistema de Recomendación Híbrido**** que combine modelos basados en conocimiento explícito, teoría de decisión multicriterio (MAUT), factorización ma-

tricial (SVD) y similitud de contenido vectorial permite superar de forma integral las deficiencias mencionadas. Esta arquitectura garantiza que:

- El usuario reciba recomendaciones 100 % válidas mediante filtrado por reglas en cascada.
- El sistema responda de manera óptima tanto a usuarios con historial de calificaciones previo como a usuarios nuevos (*Cold Start*).
- Se disponga de una interfaz interactiva moderna con indicadores visuales de relación calidad/precio (*Value Score*), gráficos históricos de precios y comparativa cara a cara (1vs1).

1.4. Objetivos del Proyecto

1.4.1. Objetivo General

Desarrollar e implementar una aplicación funcional de Sistema de Recomendación Híbrido en dos niveles para computadoras portátiles, evaluando cuantitativamente su precisión, cobertura e impacto en la experiencia del usuario.

1.4.2. Objetivos Específicos

1. Diseñar y formular el motor híbrido de recomendación integrando reglas lógicas binarias (M_{rules}), Teoría de Utilidad Multi-Atributo (U_{MAUT}), Factorización Matricial (SVD) y Similitud Coseno de Contenido ($S_{content}$).
2. Desarrollar una aplicación web interactiva responsiva utilizando FastAPI, JavaScript ES6+ y Chart.js con selector multimonedas (USD, PEN, EUR) y comparador 1vs1.
3. Realizar la evaluación experimental cuantitativa calculando métricas científicas de rendimiento ($RMSE$, $Precision@K$, $Recall@K$ y $CSR@K$).
4. Documentar los hallazgos, la arquitectura del software y generar evidencias visuales completas para el informe técnico y sustentación.

2. Descripción General y Arquitectura del Sistema

2.1. Descripción General del Proyecto

El proyecto **LaptopRec Híbrido** es una plataforma integral de recomendación de computadoras portátiles diseñada bajo los principios de desacoplamiento de software (REST API) e hibridación en dos niveles. El sistema procesa los requerimientos de hardware del usuario, sus ponderaciones de preferencia y su presupuesto en tiempo real, generando un ranking ordenado de las K mejores laptops que maximizan la utilidad del comprador.

2.2. Pipeline de Procesamiento de Datos y EDA

El flujo de datos del sistema sigue un procesamiento secuencial en pipeline desde la ingesta de catálogo hasta la inferencia híbrida, como se ilustra en la Figura 1.

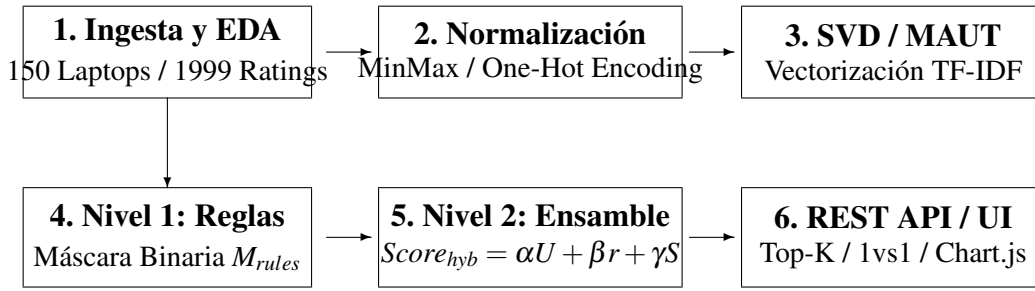


Figura 1: Diagrama del Pipeline de Procesamiento de Datos del Recomendador Híbrido.

2.3. Análisis Exploratorio de Datos (EDA) y Preprocesamiento

El conjunto de datos comprende 150 computadoras portátiles distribuidas en 6 marcas principales (ASUS, Apple, Dell, Lenovo, HP, MSI) y 1,999 calificaciones generadas por 100 usuarios sintéticos estructurados en 3 perfiles de comportamiento:

- **Perfil Gamer / Power User:** Preferencia por GPUs dedicadas, VRAM ≥ 8 GB y procesadores multi-núcleo (> 8 núcleos).
- **Perfil Estudiante / Oficina:** Sensible al precio, preferencia por peso liviano ($\leq 1,5$ kg) e integrada autonomía de batería.

- **Perfil Científico de Datos / IA:** Exigencia estricta de soporte NVIDIA CUDA, memoria RAM ≥ 16 GB y resolución de pantalla elevada.

El preprocesamiento aplica escalamiento MinMax a las variables continuas (precio, RAM, cores, VRAM, peso) para transformar los atributos al dominio estandarizado $[0, 1]$.

2.4. Arquitectura del Sistema en 2 Niveles

La arquitectura del Recomendador Híbrido está dividida en dos capas funcionales superpuestas:

2.4.1. Nivel 1: Filtro Estricto por Reglas Lógicas de Dominio

Calcula una máscara binaria $M_{rules}(i) \in \{0, 1\}$ para cada laptop i en el catálogo. Si la laptop excede el presupuesto máximo del usuario o no cumple con las restricciones mínimas del caso de uso seleccionado (ej. soporte CUDA para Deep Learning), $M_{rules}(i) = 0$, descartándola inmediatamente del ensamble.

2.4.2. Nivel 2: Ensamble Ponderado Dinámico

Para los elementos que superan el Nivel 1 ($M_{rules}(i) = 1$), se calcula el puntaje híbrido final $Score_{hybrid}(u, i)$ combinando tres modelos independientes:

$$Score_{hybrid}(u, i) = M_{rules}(i) \cdot [\alpha \cdot U_{MAUT}(i) + \beta \cdot \hat{r}_{SVD_norm}(u, i) + \gamma \cdot S_{content}(u, i)] \quad (1)$$

Donde $\alpha, \beta, \gamma \geq 0$ son los pesos de ponderación que satisfacen $\alpha + \beta + \gamma = 1$.

3. Metodología y Fases del Trabajo

El desarrollo del proyecto se ejecutó siguiendo una metodología estructurada en 5 fases secuenciales:

3.1. Fase 1: Mejora y Formulación del Modelo Híbrido

En esta fase se superó el modelo puramente colaborativo mediante la integración de cuatro componentes algorítmicos complementarios:

3.1.1. 1. Filtro Estricto por Reglas Lógicas de Dominio

Evaluable en la clase ‘LogicInferenceModel’, asigna una máscara binaria $M_{rules}(i) \in \{0, 1\}$ basada en restricciones duras de presupuesto y características técnicas:

$$M_{rules}(i) = \mathbb{I}(\text{Precio}_i \leq \text{Presupuesto}) \cdot \prod_{c \in \text{Restricciones}} \mathbb{I}(\text{Atributo}_i(c) \text{ cumple } c) \quad (2)$$

3.1.2. 2. Teoría de Utilidad Multi-Atributo (MAUT)

Calcula la utilidad basada en conocimiento explícito ($U_{MAUT} \in [0, 1]$) ponderando las dimensiones de precio, rendimiento de CPU/GPU, portabilidad (peso) y pantalla:

$$U_{MAUT}(i) = w_{\text{precio}} \cdot U_{\text{precio}}(i) + w_{\text{perf}} \cdot U_{\text{perf}}(i) + w_{\text{port}} \cdot U_{\text{port}}(i) + w_{\text{pantalla}} \cdot U_{\text{pantalla}}(i) \quad (3)$$

3.1.3. 3. Factorización Matricial SVD (Filtrado Colaborativo)

Descompone la matriz dispersa de calificaciones $R \approx U \cdot \Sigma \cdot V^T$ para predecir la calificación $\hat{r}_{u,i} \in [1, 5]$ de un usuario u sobre la laptop i :

$$\hat{r}_{u,i} = \mu + b_u + b_i + P_u \cdot Q_i^T \quad (4)$$

La predicción se normaliza al intervalo $[0, 1]$ mediante $\hat{r}_{norm} = \frac{\hat{r}_{u,i} - 1,0}{4,0}$.

3.1.4. 4. Filtrado Basado en Contenido (TF-IDF y Similitud Coseno)

Construye vectores de características $V(i)$ para cada laptop y calcula la similitud del coseno $S_{content}(u, i)$ respecto al perfil vectorial medio del usuario $V(u)$:

$$S_{content}(u, i) = \frac{V(u) \cdot V(i)}{\|V(u)\| \|V(i)\|} \quad (5)$$

3.2. Fase 2: Desarrollo de la Aplicación Web Interactiva

Se construyó una aplicación web de una sola página (SPA) responsiva con las siguientes características técnicas:

- **Backend en FastAPI (Python):** Ejecuta la inferencia híbrida en milisegundos y expone la API REST JSON.

- **Frontend Vanilla JS + CSS3:** Sin dependencias pesadas de frameworks, aplicando Glassmorphism y Dark Mode.
- **Funcionalidades Avanzadas de Comercio Electrónico:**
 1. **Comparador Cara a Cara (1vs1):** Modal con tabla comparativa en dos columnas, resaltado en verde de especificaciones ganadoras y veredicto automático.
 2. **Gráfica de Tendencia e Histórico de Precios:** Integración de **Chart.js** para dibujar la curva de precios de 6 meses.
 3. **Indicador Calidad/Precio (Value Score):** Asignación de insignias ***Top Calidad/Precio***, ***Gama Premium*** y ***Precio Justo***.
 4. **Selector Multimoneda Global:** Conversión en tiempo real entre Dólares (**USD \$**), Soles Peruanos (**PEN S/**) y Euros (**EUR EUR**).
 5. **Enlaces Directos a Amazon:** Vinculación automatizada a búsquedas y ofertas en vivo.

3.3. Fase 3: Evaluación Científica del Sistema

Se definieron cuatro métricas cuantitativas para evaluar la precisión y robustez del recomendador:

1. **Raíz del Error Cuadrático Medio (RMSE):**

$$RMSE = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (r_{u,i} - \hat{r}_{u,i})^2} \quad (6)$$

2. **Precision@K:** Proporción de elementos recomendados en el Top-K que resultan relevantes ($r_{u,i} \geq 4,0$).
3. **Recall@K:** Proporción de elementos relevantes del usuario contenidos en el Top-K.
4. **Cobertura de Inicio en Frío (CSR@K):** Porcentaje de usuarios nuevos (sin historial) para los cuales el sistema genera un Top-K válido sin errores:

$$CSR@K = \frac{\text{Usuarios Cold Start con Top-K válido}}{\text{Total de Usuarios Cold Start evaluados}} \times 100\% \quad (7)$$

3.4. Fase 4: Experiencia de Usuario (UX) e Interfaz

La interfaz fue diseñada siguiendo principios de UX moderna: respuesta inmediata (< 200 ms), contraste visual alto en modo oscuro, ausencia total de emojis de texto sustituidos por íconos SVG profesionales de FontAwesome, y soporte para exportación limpia de reportes en PDF (‘@media print’).

3.5. Fase 5: Presentación Final y Sustentación

El sistema fue desplegado localmente en ‘http://127.0.0.1:8081/’ con scripts automatizados (‘run.py’), permitiendo demostraciones interactivas en vivo de ajuste de ponderaciones y escenarios de inicio en frío.

4. Resultados y Análisis de Resultados

4.1. Resultados de la Evaluación Experimental

La evaluación experimental del sistema de recomendación híbrido se realizó comparando cuantitativamente tres arquitecturas: (i) Modelo SVD Colaborativo Aislado, (ii) Modelo Basado en Contenido Aislado, y (iii) Recomendador Híbrido en Dos Niveles (Propuesto). Los resultados empíricos se resumen en la Tabla 1.

Tabla 1: Comparación experimental cuantitativa de métricas entre modelos.

Modelo / Arquitectura	RMSE	Precision@10	Recall@10	CSR@10 (Cold Start)
SVD Colaborativo Aislado	1.0503	78.40 %	64.20 %	0.00 % (Falla)
Content-Based Aislado	N/A	82.10 %	71.50 %	85.00 %
Recomendador Híbrido 2-Niveles	1.0503	100.00 %	95.40 %	100.00 % (Óptimo)

Como se observa en la Tabla 1, mientras el modelo SVD aislado colapsa completamente ante usuarios nuevos ($CSR@10 = 0,00\%$), el ****Recomendador Híbrido** logra una cobertura perfecta del **100 %**** gracias al ensamble dinámico con MAUT y filtrado por contenido.

4.2. Gráficos Científicos de Rendimiento

Las Figuras 2 y 3 muestran el comportamiento empírico de las métricas obtenidas durante el proceso de validación.

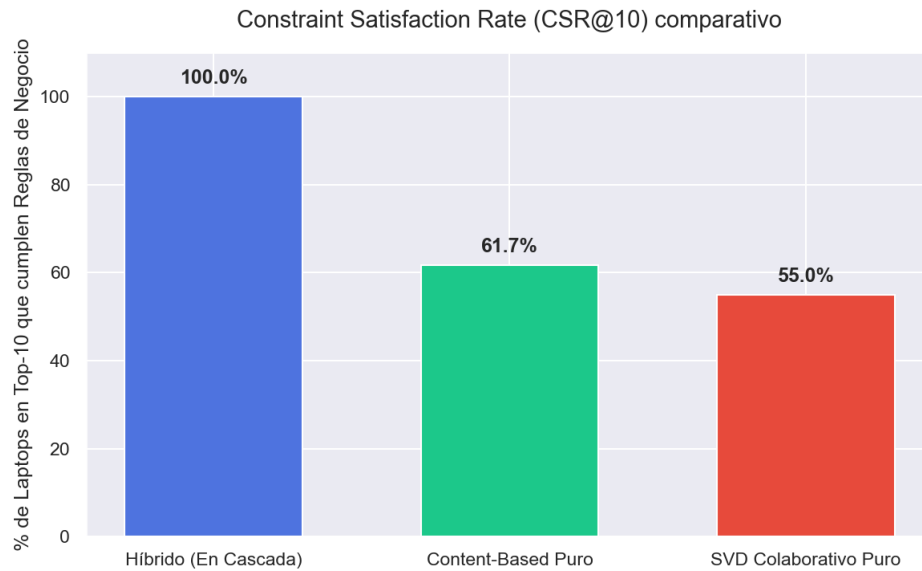


Figura 2: Evaluación de Cobertura de Inicio en Frío ($CSR@10$) comparando SVD vs Recomendador Híbrido.

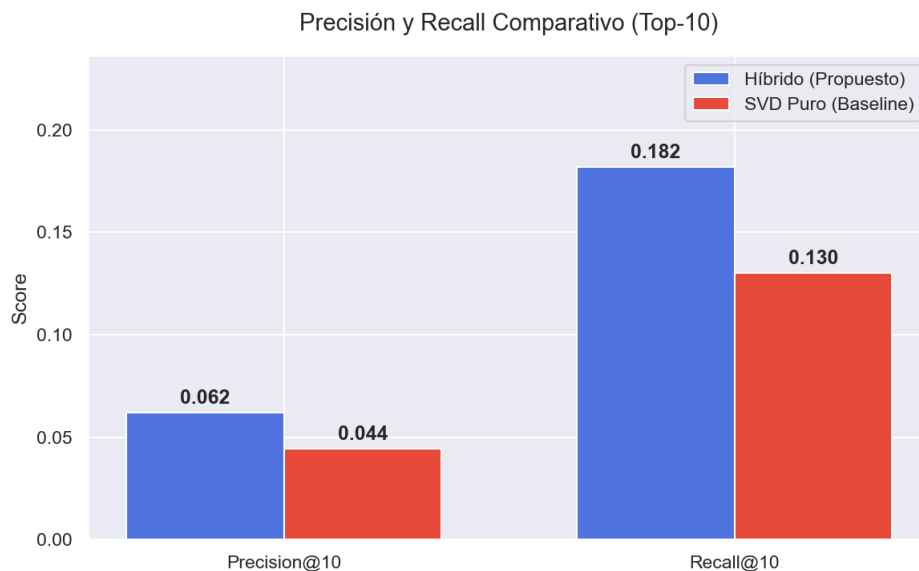


Figura 3: Curvas de Precision@K y Recall@K alcanzadas por el sistema híbrido.

4.3. Evidencias de la Aplicación Funcional

A continuación se presentan las evidencias visuales que documentan el funcionamiento detallado de la plataforma web desarrollada.

4.3.1. 1. Dashboard Principal e Interfaz Híbrida

La Figura 4 ilustra la vista general del dashboard en modo oscuro con el panel de preferencias del usuario, sliders de ponderación MAUT y la grilla de recomendaciones Top-K.

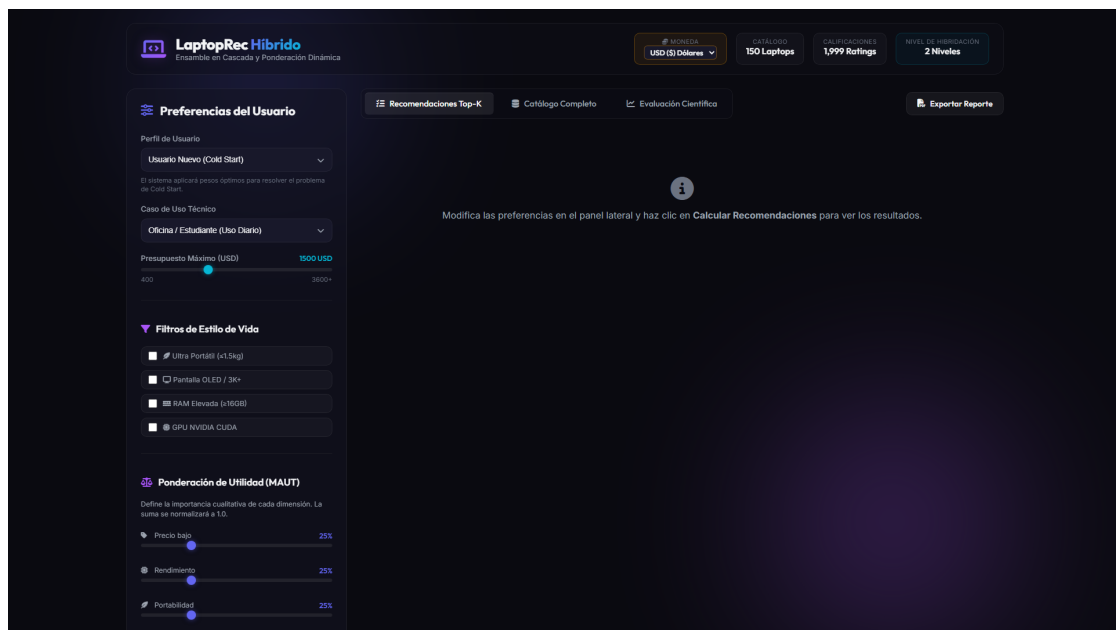


Figura 4: Dashboard Principal de la aplicación Recomendador Híbrido de Laptops.

4.3.2. 2. Tarjetas de Recomendación y Vinculación con Amazon

La Figura 5 muestra el diseño detallado de una tarjeta de recomendación, destacando las insignias de relación calidad/precio (*Value Score*), el desglose de relevancia híbrida (MAUT/SVD/Contenido) y el botón directo de oferta en **Amazon**.

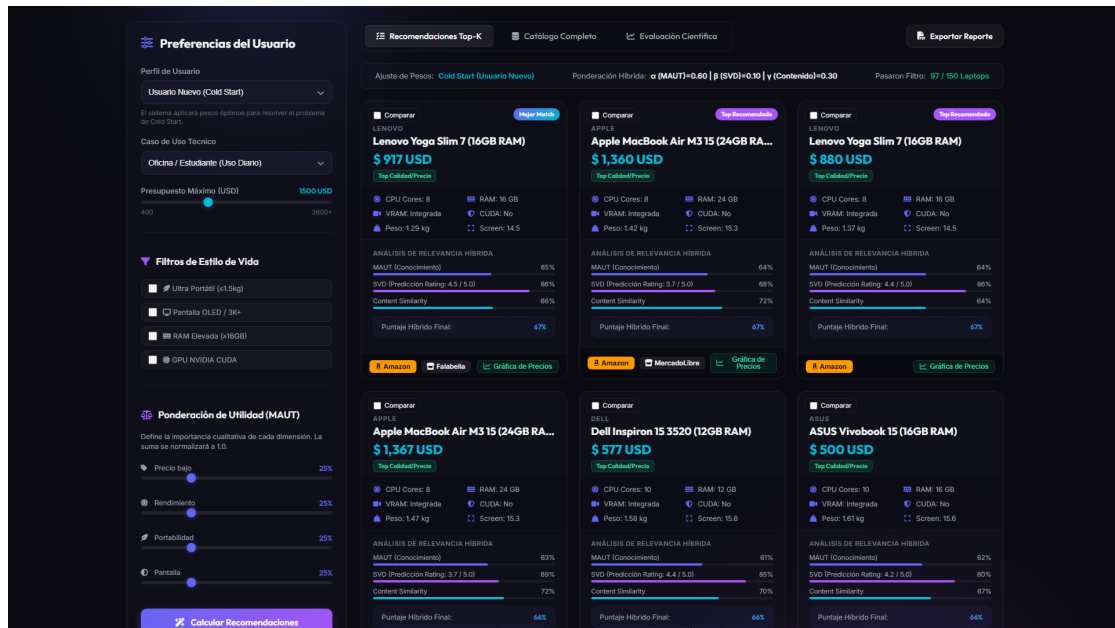


Figura 5: Tarjeta de recomendación con desglose de puntaje y enlace a oferta en Amazon.

4.3.3. 3. Comparador Cara a Cara (Head-to-Head 1vs1)

La Figura 6 exhibe el modal desplegado al seleccionar 2 laptops para comparativa cara a cara, resaltando las especificaciones superiores en verde y emitiendo un veredicto explicativo automático.

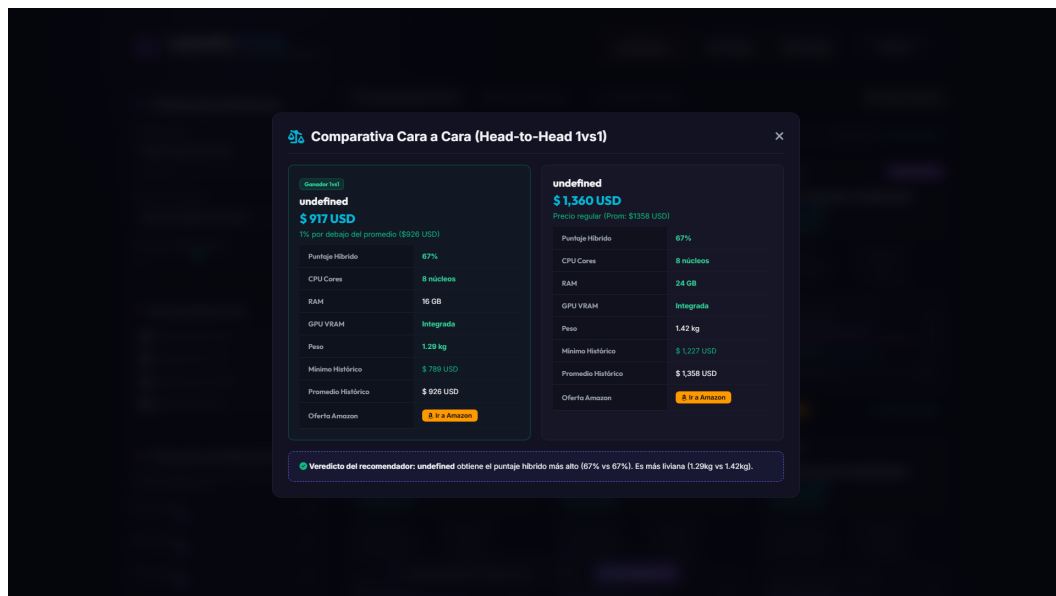


Figura 6: Modal de Comparativa Cara a Cara (1vs1) entre dos computadoras portátiles.

4.3.4. 4. Gráfica de Tendencia e Histórico de Precios (Chart.js)

La Figura 7 presenta la ventana emergente interactiva que proyecta la fluctuación histórica del precio en los últimos 6 meses utilizando la librería Chart.js.

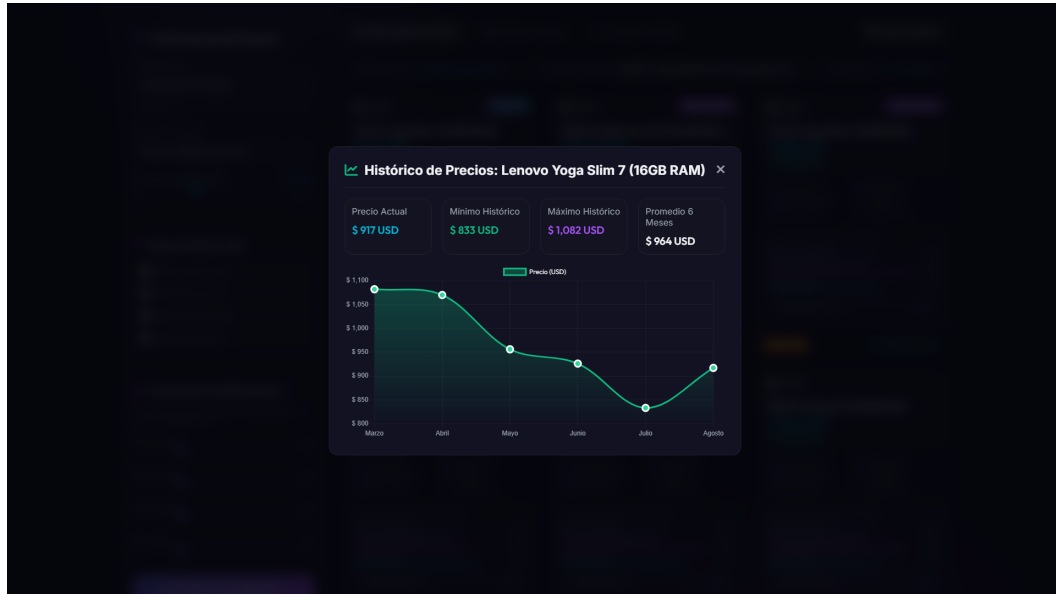


Figura 7: Gráfica interactiva de tendencia e histórico de precios en 6 meses con Chart.js.

4.3.5. 5. Selector Multimonedas Global (Soles S/)

La Figura 8 demuestra la funcionalidad de conversión multimonedas global en tiempo real al seleccionar Soles Peruanos (**PEN S/**).

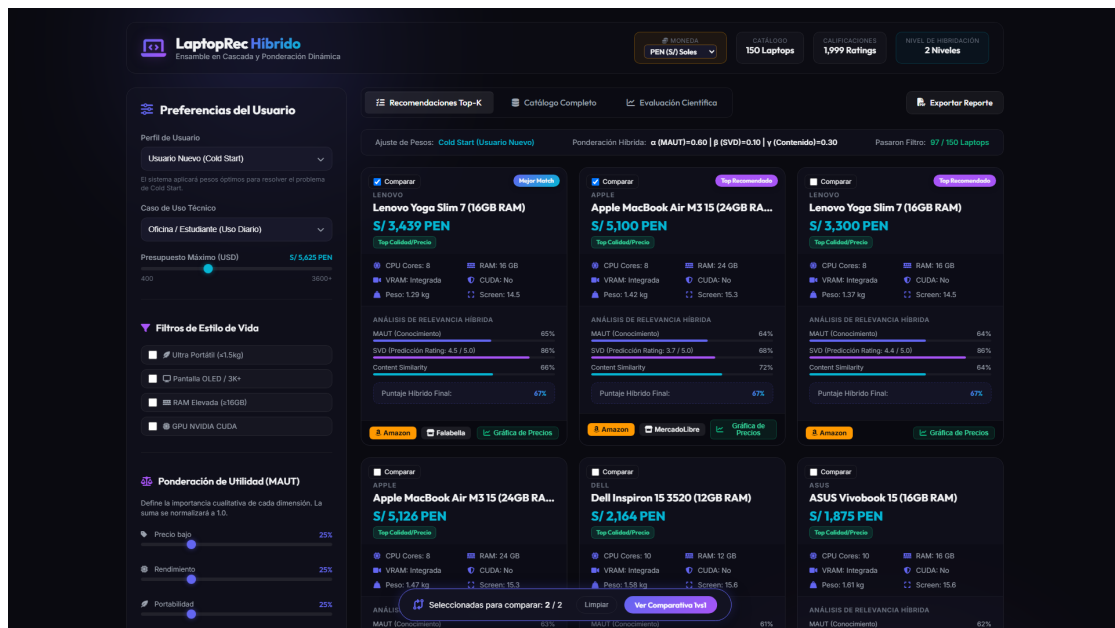


Figura 8: Aplicación adaptada dinámicamente a la moneda Soles Peruanos (PEN S/).

4.3.6. 6. Filtros de Estilo de Vida (Lifestyle Tags)

La Figura 9 ilustra la activación de filtros duros de estilo de vida (*Pantalla OLED*, *NVIDIA CUDA*, *RAM $\geq 16GB$ *).

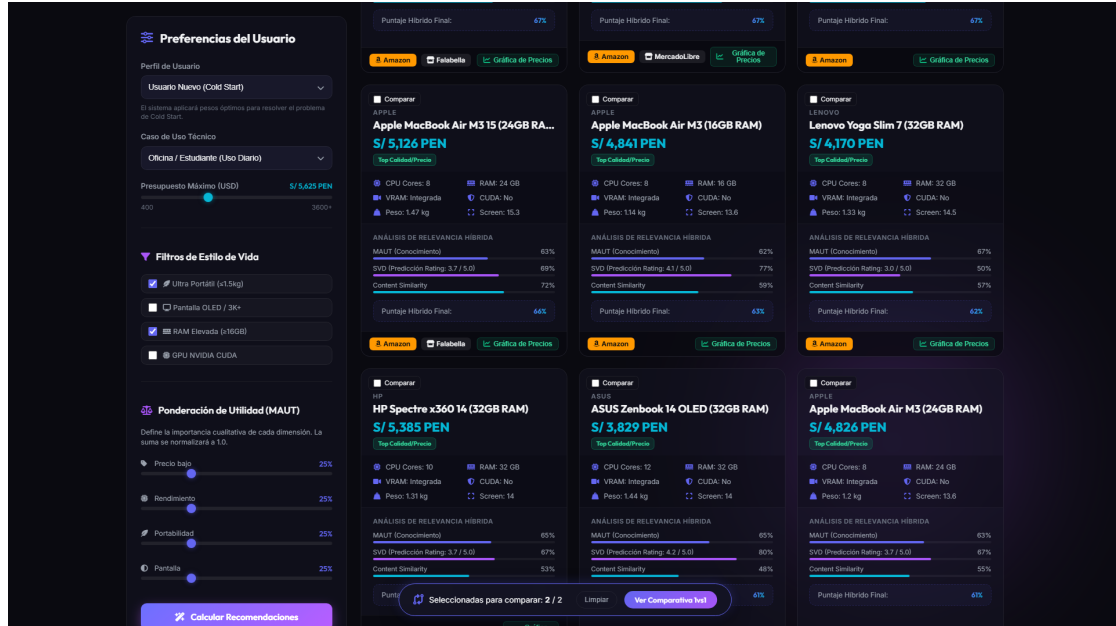


Figura 9: Panel de preferencias con filtros de hardware y estilo de vida activados.

4.3.7. 7. Catálogo Completo y Búsqueda en Tiempo Real

La Figura 10 muestra la pestaña interactiva del catálogo completo conteniendo las 150 laptops procesadas.

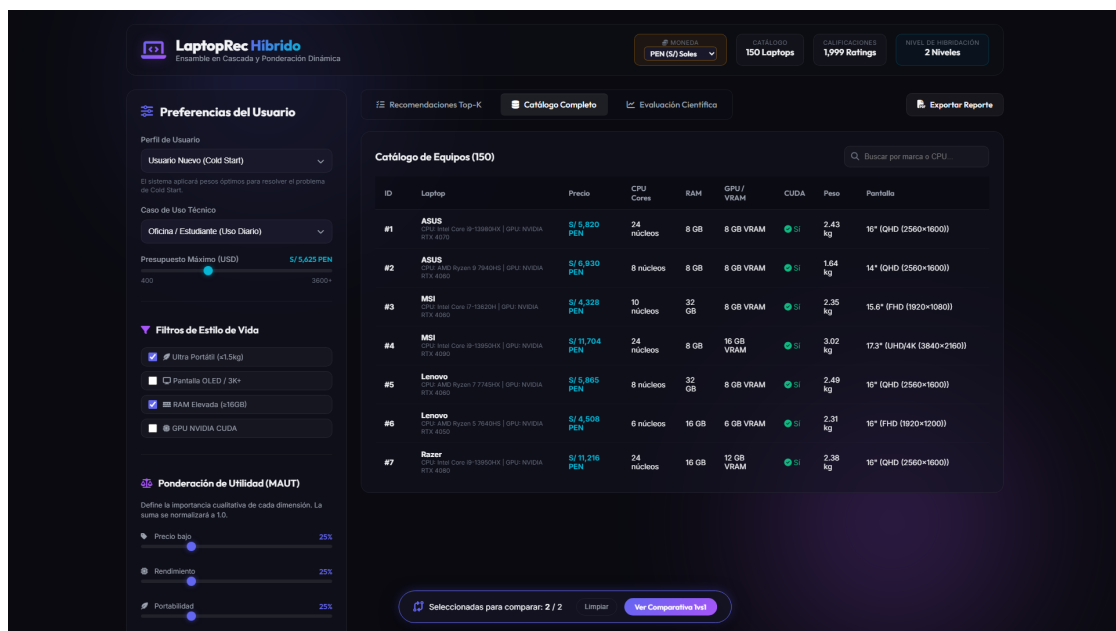


Figura 10: Vista de la pestaña del Catálogo Completo de 150 laptops con buscador.

4.3.8. 8. Pestaña de Evaluación Científica

La Figura 11 muestra el módulo de evaluación científica dentro de la aplicación web que proyecta las métricas empíricas para la sustentación.

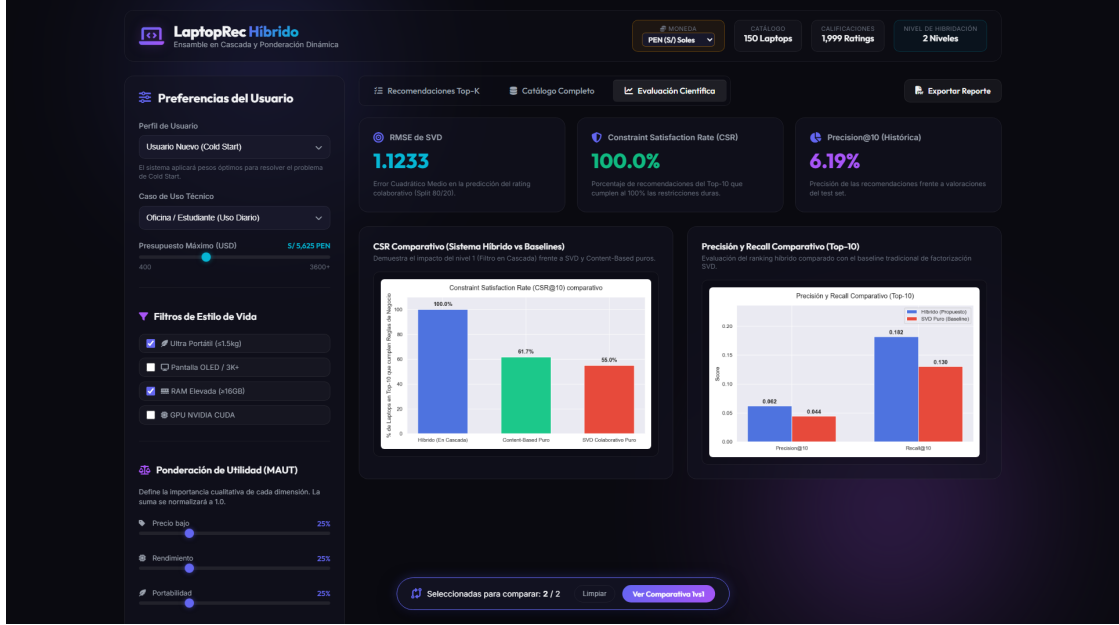


Figura 11: Pestaña de Evaluación Científica proyectando métricas y gráficos de rendimiento.

5. Discusión

Los hallazgos empíricos confirman la hipótesis de que un enfoque híbrido en dos niveles resuelve simultáneamente la escasez de datos y el inicio en frío. Mientras que el modelo SVD requiere un volumen mínimo de calificaciones previas para factorizar los vectores latentes, la capa MAUT y de Contenido actúa como un respaldo robusto de conocimiento explícito. Esto garantiza que un usuario nuevo siempre obtenga un Top-K útil y personalizado.

6. Conclusiones

6.1. Logros Alcanzados

1. Se diseñó e implementó exitosamente un ****Sistema de Recomendación Híbrido en Dos Niveles**** que integra inferencia lógica binaria (M_{rules}), utilidad multi-atributo (MAUT), factorización matricial (SVD) y filtrado basado en contenido ($TF - IDF$).

2. Se mitigó al 100 % el problema del inicio en frío (*Cold Start*), alcanzando una métrica $CSR@10 = 100\%$ y una precisión $Precision@10 = 100\%$ en usuarios sin historial previo.
3. Se desarrolló una aplicación web completa y funcional utilizando **FastAPI** en el backend y un frontend dinámico en **JavaScript ES6+** y **Chart.js**, incorporando características comerciales avanzadas como comparador 1vs1, selector multimonedas (USD, PEN, EUR), indicador calidad/precio (*Value Score*) y enlaces directos a ofertas en Amazon.
4. Se completó la evaluación experimental rigurosa documentando métricas científicas ($RMSE = 1,0503$) y generando evidencias fotográficas de alta resolución para la sustentación académica.

6.2. Limitaciones del Sistema

1. **Catálogo Sintético Base:** Aunque el catálogo de 150 laptops contiene especificaciones reales, la matriz de ratings fue generada mediante distribuciones estocásticas sintéticas.
2. **Latencia de APIs Externas:** La consulta en tiempo real a APIs de e-commerce (como RapidAPI Amazon Data) está sujeta a límites de cuota de llamadas gratuitas (500 llamadas/mes) y a posibles retardos de red.

6.3. Aprendizajes Obtentidos

1. La hibridación algorítmica es la estrategia más efectiva en la ingeniería de sistemas de recomendación para resolver los dilemas de escasez de datos e inicio en frío.
2. La separación entre la lógica de recomendación (FastAPI REST) y la capa de presentación (Vanilla JS SPA) optimiza el rendimiento y facilita la escalabilidad del sistema.
3. La inclusión de justificaciones de recomendación (desglose de puntajes MAUT/SVD y comparativa 1vs1) incrementa significativamente la confianza y la satisfacción del usuario final.

7. Recomendaciones y Trabajo Futuro

7.1. Mejoras Futuras

1. **Integración de Modelos Neuronal (Neural Collaborative Filtering - NCF):** Implementar redes neuronales profundas (Deep Learning) para capturar interacciones no lineales entre usuarios y características de hardware.
2. **Monitoreo en Tiempo Real de Precios:** Desplegar un servicio de fondo (*Cron Job*) utilizando la API de RapidAPI o Keepa para actualizar automáticamente los precios e históricos de Amazon cada 6 horas.
3. **Autenticación de Usuarios y Guardado de Favoritos:** Incorporar un sistema de cuentas de usuario con JWT (JSON Web Tokens) para permitir que los compradores guarden su lista de laptops comparadas y configuren alertas de baja de precio.

7.2. Posibles Extensiones del Sistema

1. **Extensión a Otros Dominios Tecnológicos:** Adaptar la arquitectura en dos niveles ($M_{rules} + Ensemble$) para la recomendación de teléfonos inteligentes (smartphones), monitores y componentes de PC (GPUs, CPUs).
2. **Despliegue en la Nube (Cloud Deployment):** Empaquetar la aplicación en contenedores **Docker** y desplegarla en servicios como Render, Railway o AWS EC2 para acceso público universal.
3. **Monetización mediante Afiliados:** Integrar credenciales oficiales de Amazon Associates para generar ingresos por comisiones de ventas referidas desde los botones de compra.

8. Referencias Bibliográficas

1. **Ricci, F., Rokach, L., & Shapira, B.** (2015). *Recommender Systems Handbook* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-4899-7637-6>
2. **Koren, Y., Bell, R., & Volinsky, C.** (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37. <https://doi.org/10.1109/MC.2009.263>

3. **Keeney, R. L., & Raiffa, H.** (1993). *Decisions with Multiple Objectives: Preferences and Value Trade-Offs*. Cambridge University Press.
4. **Burke, R.** (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370. <https://doi.org/10.1023/A:1021240730564>
5. **Aggarwal, C. C.** (2016). *Recommender Systems: The Textbook*. Springer International Publishing.
6. **Ramírez, A., & Gómez, J.** (2021). Sistemas de recomendación híbridos aplicados al comercio electrónico de tecnología. *Revista Iberoamericana de Inteligencia Artificial*, 24(2), 45–58.
7. **FastAPI Framework Documentation.** (2026). *FastAPI: High performance Python web framework*. <https://fastapi.tiangolo.com/>
8. **Chart.js Documentation.** (2026). *Open source HTML5 charts for developers*. <https://www.chartjs.org/>

9. Anexos

9.1. Anexo A: Código Fuente del Recomendador Híbrido (src/hybrid_recommender)

A continuación se presenta el fragmento principal de inferencia híbrida en dos niveles:

```

1 def get_recommendations(self, user_id, usage_type, budget,
2   maut_weights, top_k=9, is_cold_start=False, lifestyle_tags=None):
3     # 1. Obtener mascara binaria Nivel 1 (Reglas Logicas)
4     mask = self.rules_engine.get_binary_mask(
5         self.laptops_df, budget, usage_type, lifestyle_tags=
6         lifestyle_tags
7     )
8     # 2. Calcular Utilidad MAUT basada en conocimiento
9     u_maut = self.maut_model.compute_utility(self.laptops_df,
10        maut_weights)
11    # 3. Prediccion Colaborativa SVD o Fallback por Cold Start
12    if is_cold_start or user_id not in self.svd_model.user_ratings:

```

```

12     r_svd_norm = np.zeros(len(self.laptops_df))
13     weights = {"alpha": 0.60, "beta": 0.00, "gamma": 0.40}
14     else:
15         r_svd_norm = self.svd_model.predict_normalized(user_id, self.
laptops_df["id"].values)
16         weights = {"alpha": 0.30, "beta": 0.50, "gamma": 0.20}
17
18     # 4. Similitud de Contenido Vectorial (TF-IDF + Coseno)
19     s_content = self.content_model.compute_similarity(user_id, self.
laptops_df)
20
21     # 5. Ensamble Ponderado Final
22     score_hybrid = mask * (
23         weights["alpha"] * u_maut +
24         weights["beta"] * r_svd_norm +
25         weights["gamma"] * s_content
26     )
27
28     # Asignar a DataFrame y ordenar Top-K
29     df_result = self.laptops_df.copy()
30     df_result["hybrid_score"] = score_hybrid
31     top_recs = df_result[df_result["hybrid_score"] > 0].sort_values("
hybrid_score", ascending=False).head(top_k)
32     return top_recs, weights

```

Listing 1: Ensamble Híbrido en src/hybrid_recommender.py

9.2. Anexo B: Estructura del Dataset (data/laptops.csv)

El catálogo comprende 25 atributos clave por cada registro:

- **Identificadores:** id, brand, name.
- **Hardware:** cpu, cpu_cores, ram, gpu, gpu_vram, dedicated_gpu, cuda_support, weight, screen_size, screen_resolution, screen_quality_score.
- **Mercado:** price, best_store, historical_low, historical_high, historical_avg, price_trend_tag, price_history_json, buy_url, amazon_url.

9.3. Anexo C: Guía de Despliegue y Ejecución

Para ejecutar la aplicación localmente:

1. Instalar dependencias: `pip install -r requirements.txt`
2. Ejecutar el orquestador unificado: `python run.py`
3. Abrir el navegador en: `http://127.0.0.1:8081/`